

Практическая работа по Информатике

Подпрограммы. Рекурсия

Рассмотрим программу для печати на экране фигуры бриллианта с задаваемым размером по высоте, вводимое целое должно быть нечётным. Сначала оценим вариант без использования подпрограмм (этот исходный текст можно найти в файле /home/method/diamond.pas):

```
program diamond;
var
    n, k, h, i: integer;
begin
    repeat
        write('Enter the diamond's height (positive odd): ');
        readln(h)
    until (h > 0) and (h mod 2 = 1);
    n := h div 2;
    for k := 1 to n + 1 do
    begin
        for i := 1 to n + 1 - k do
            write(' ');
        write('*');
        if k > 1 then
        begin
            for i := 1 to 2*k - 3 do
                write(' ');
            write('*');
        end;
        writeln;
    end;
    for k := n downto 1 do
    begin
        for i := 1 to n + 1 - k do
            write(' ');
        write('*');
        if k > 1 then
        begin
            for i := 1 to 2*k - 3 do
                write(' ');
            write('*');
        end;
        writeln;
    end
end.
```

При вводе, например, числа 7 получим такой результат работы этой программы:
Enter the diamond's height (positive odd): 7

```
  *
 * *
*   *
*   *
* *
 *
```

Теперь подумаем, как нам оптимизировать текст этой программы. Мы видим два практически одинаковых цикла, первый из которых рисует верхнюю часть бриллианта, а второй – нижнюю. Кроме того, внутри каждого из этих циклов повторяется ещё и внутренний, вложенный цикл, который выводит нужное число пробелов: сначала пробелы до первой звёздочки, а затем пробелы до второй звёздочки. Начнём оптимизацию с элементарных операций. Попробуем вынести этот внутренний цикл в подпрограмму. По правилам Паскаля для этого надо использовать процедуру. А так как число пробелов у нас может быть разным, то нашу процедуру снабдим параметром, который указывается в круглых скобках после имени процедуры. В этом случае параметр будет всего один – количество пробелов, которые надо напечатать. Процедуру назовём PrintSpaces, а параметр – count; заголовок процедуры будет такой:

```
procedure PrintSpaces(count: integer);
```

Вспомним, что для печати заданного количества пробелов нам нужен цикл for, а в нём нужна будет переменная типа integer в роли счётчика цикла. Так как эта переменная – частная деталь реализации нашей процедуры, то её нужно сделать локальной переменной, то есть такой, которая не видна за пределами процедуры. Для этого у процедуры есть собственная секция описаний, аналогичная основной программе. Полностью наша процедура будет выглядеть так:

```
procedure PrintSpaces(count: integer);
var
    i: integer;
begin
    for i := 1 to count do
        write(' ')
    end;
```

Заметим, что имена **i** и **count** в этой процедуре локальные, то есть они не оказывают никакого влияния на остальную программу; мы можем в любом месте описать переменные или константы с такими же именами, даже другого типа, это не приведёт к плохим последствиям.

Теперь, после описания процедуры, мы можем вызывать её в любом месте программы нужное число раз, написав, например, PrintSpaces(m), в результате будет напечатано m пробелов.

На следующем шаге оптимизации программы попробуем вынести в процедуру печать строки со звёздочками. Очередная, k-я строка всей фигуры, печатается так: сначала надо напечатать n+1-k пробелов, потом звёздочку, далее, если k > 1, то напечатать 2k-3 пробела и ещё одну звёздочку, и в конце перевести строку. Как видим, для выполнения этих шагов надо знать две величины: k и n, причём эта ситуация повторяется и для печати верхней части и нижней части фигуры.

Вынесем печать отдельной строки нашей фигуры в процедуру, которую назовём PrintLineOfDiamond; числа k и n мы передадим в процедуру через параметры. После замены тел обоих циклов вызовами этой процедуры главная программа станет совсем короткой:

```
program diamond;

procedure PrintSpaces(count: integer);
var
    i: integer;
begin
    for i := 1 to count do
        write(' ')
    end;
```

```

procedure PrintLineOfDiamond(k, n: integer);
begin
    PrintSpaces(n + 1 - k);
    write('*');
    if k > 1 then
    begin
        PrintSpaces(2*k - 3);
        write('*')
    end;
    writeln
end;

var
    n, k, h: integer;
begin
    repeat
        write('Enter the diamond''s height (positive odd): ');
        readln(h)
    until (h > 0) and (h mod 2 = 1);
    n := h div 2;
    for k := 1 to n + 1 do
        PrintLineOfDiamond(k, n);
    for k := n downto 1 do
        PrintLineOfDiamond(k, n)
    end.
end.

```

Рекурсия

Некоторые алгоритмы сильно упрощаются, если подпрограмма может вызвать саму себя. Действительно, вспомним о том, что локальные переменные создаются в памяти только на время вызова подпрограммы.

Для простого примера перепишем процедуру PrintSpaces с использованием рекурсии вместо цикла. Для начала заметим, что, во-первых, случай, когда нужное количество символов равно нулю – вырожденный, при котором делать ничего не надо; во-вторых, если случай попался не вырожденный, то напечатать n символов – это то же самое, что напечатать сначала один символ, а потом $(n - 1)$ символов, при этом задача «напечатать $(n - 1)$ символов» вполне годится в качестве «чуть более простого» случая той же самой задачи. Рекурсивная реализация процедуры будет выглядеть так:

```

procedure PrintChars(ch:char; count: integer);
begin
    if count > 0 then
    begin
        write(ch);
        PrintChars(ch, count-1)
    end
end;
end;

```

Задание

Измените программу Diamond так, чтобы пользователь мог вводить символ для печати фигуры. Используйте рекурсивную процедуру PrintChars. Можно использовать и другие процедуры и функции.